Міністерство освіти і науки України
Національний університет «Львівська політехніка»
Кафедра інформаційних систем та мереж

ЗВІТ

до лабораторної роботи № 7.1

з дисципліни «Алгоритмізація та програмування»

на тему: «Пошук заданих елементів та впорядкування рядків / стовпчиків матриці»

(Рекурсивний спосіб)

Виконав:
студент групи РІ-13
Шах Владислав Ігорьович

Варіант № 32

Перевірив:
Григорович В. Г.

Львів – 2026

1. Мета роботи

Сформувати практичні навички розробки програм опрацювання матриць із використанням виключно рекурсивного способу; реалізувати алгоритми формування матриці, виведення її елементів на екран, лексикографічного впорядкування стовпчиків, пошуку заданих елементів за критерієм варіанта, обчислення їх кількості та суми із подальшою заміною нулями без застосування жодної ітераційної конструкції (циклів for, while, do..while).

2. Умова завдання (Варіант 32)

Сформувати матрицю цілих чисел розмірністю 7 × 9 (7 рядків, 9 стовпчиків) із діапазону [-12, 37].
Критерій для варіанта 32: знайти всі додатні елементи або крім кратних 3 (тобто елементи, для яких виконується умова: x > 0 || !(x % 3 == 0)).
Обчислити суму (S) та кількість (k) знайдених елементів, після чого замінити їх нулями.
Впорядкувати стовпчики матриці за зростанням ключів: спочатку за елементами 0-го рядка; при їх рівності — за елементами 1-го рядка; при їх рівності — за елементами 2-го рядка (напрям ключових елементів: → → →).

Приклади розв'язання лабораторних завдань

Умова завдання

Написати програму, яка:

1. за допомогою генератора випадкових чисел формує вказану матрицю – так, значення її елементів належать заданому діапазону. Обчислити кількість та с тих елементів, які задовольняють вказаному критерію; а також замінити нулями елементи;
2. впорядковує матрицю, переставляючи місцями її рядки / стовпчики відповідно вказаних ключів впорядкування.

Пояснити призначення змінних.

Необхідно:

- (завдання 1 і 2) сформувати матрицю;
- (завдання 1 і 2) вивести її на екран у вигляді таблиці, використовуючи форма виведення;

Рис. 1.1. Загальні вимоги до виконання лабораторної роботи № 7.1 із посібника

Варіант 32.

Діапазон значень елементів матриці: (-12, …, 37)

Матриця *R* розмірністю (7 × 9) із елементів цілого типу.

Ключі впорядкування: → → →

– переставляються місцями стовпчики матриці – так, щоб елементи першого ря стали впорядкованими за зростанням;
– якщо в першому рядку наявні однакові елементи, то переставляються стовпч матриці – так, щоб для таких значень в першому рядку елементи другого ря стали впорядкованими за зростанням;
– якщо в першому і другому рядках наявні однакові елементи, то переставляют стовпчики матриці – так, щоб для таких значень в першому і другому ряд елементи третього рядка стали впорядкованими за зростанням.

Рис. 1.2. Індивідуальне завдання для варіанта № 32 (Лабораторна робота № 7.1)

3. Структурна схема алгоритму

Згідно з методичними вказівками та вимогами до структури програми, взаємодія між підпрограмами відображена у вигляді структурної схеми (ієрархії викликів функцій). Оскільки за вимогою завдання всі алгоритми реалізовано виключно рекурсивним способом, функції обходу матриці не містять внутрішніх циклів, а здійснюють рекурсивний самовиклик для переходу до наступного індексу (позначено спеціальними круговими стрілками рекурсії «рек.»):

• main() — головна координаційна функція, що виділяє пам'ять, викликає функції обробки та друкує підсумкові результати.
• CreateRows() та CreateRow() — дворівнева взаємна рекурсія формування псевдовипадкових елементів матриці.
• PrintRows() та PrintRow() — дворівнева рекурсія форматованого виведення матриці на консоль.
• SortCols() та SortColsPass() — рекурсивний аналог сортування обміном (bubble sort) для стовпчиків.
• CompareCols() — рекурсивне лексикографічне зіставлення ключових елементів двох стовпчиків (рядки 0, 1, 2).
• SwapCols() та SwapElements() — рекурсивний поелементний обмін двох сусідніх стовпчиків.
• CalcRows() та CalcRow() — рекурсивне обчислення суми S, кількості k та занулення елементів за функцією-предикатом Criterion().

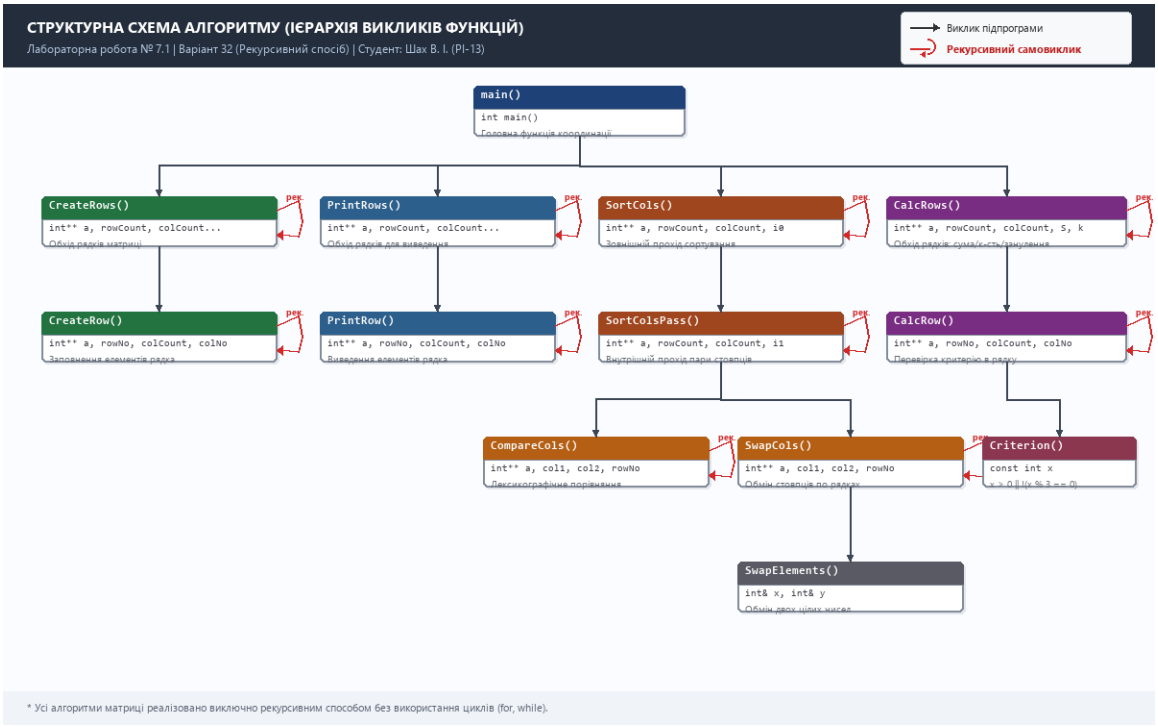


Рис. 1.3. Структурна схема алгоритму програми ЛР № 7.1 (ієрархія викликів та рекурсивні зв'язки)

4. Текст програми (C++)

Повний вихідний текст програми (файл LABA-7.1.cpp):

```
// LABA-7.1.cpp
// Шах Владислав Ігорьович
// Лабораторна робота № 7.1
// Пошук заданих елементів та впорядкування рядків / стовпчиків матриці.
// Варіант 32 (Рекурсивний спосіб)

#include <iostream>
#include <iomanip>
#include <time.h>

using namespace std;

// Рекурсивне заповнення одного рядка матриці випадковими числами
void CreateRow(int** a, const int rowNo, const int colCount, const int Low, const int High, int colNo)
{
    a[rowNo][colNo] = Low + rand() % (High - Low + 1);
    if (colNo < colCount - 1)
        CreateRow(a, rowNo, colCount, Low, High, colNo + 1);
}

// Рекурсивний обхід усіх рядків для заповнення матриці
void CreateRows(int** a, const int rowCount, const int colCount, const int Low, const int High, int
rowNo)
{
    CreateRow(a, rowNo, colCount, Low, High, 0);
    if (rowNo < rowCount - 1)
        CreateRows(a, rowCount, colCount, Low, High, rowNo + 1);
}

// Рекурсивне виведення одного рядка матриці на екран
```

```
void PrintRow(int** a, const int rowNo, const int colCount, int colNo)
{
    cout << setw(5) << a[rowNo][colNo];
    if (colNo < colCount - 1)
        PrintRow(a, rowNo, colCount, colNo + 1);
    else
        cout << endl;
}

// Рекурсивний обхід усіх рядків для виведення матриці
void PrintRows(int** a, const int rowCount, const int colCount, int rowNo)
{
    PrintRow(a, rowNo, colCount, 0);
    if (rowNo < rowCount - 1)
        PrintRows(a, rowCount, colCount, rowNo + 1);
    else
        cout << endl;
}

// Перевірка критерію варіанта 32:
// всі додатні або крім кратних 3 (тобто додатні АБО не кратні 3)
bool Criterion(int x)
{
    return (x > 0 || !(x % 3 == 0));
}

// Рекурсивне опрацювання елементів одного рядка:
// обчислення суми, кількості та заміни елементів нулями
void CalcRow(int** a, const int rowNo, const int colCount, int colNo, int& S, int& k)
{
    if (colNo >= colCount)
        return;

    if (Criterion(a[rowNo][colNo]))
    {
        S += a[rowNo][colNo];
        k++;
        a[rowNo][colNo] = 0;
    }
    CalcRow(a, rowNo, colCount, colNo + 1, S, k);
}

// Рекурсивний обхід усіх рядків для обчислення S і k та заміни нулями
void CalcRows(int** a, const int rowCount, const int colCount, int rowNo, int& S, int& k)
{
    if (rowNo >= rowCount)
        return;

    CalcRow(a, rowNo, colCount, 0, S, k);
    CalcRows(a, rowCount, colCount, rowNo + 1, S, k);
}

// Рекурсивний обмін елементів двох стовпців для одного рядка
void ChangeColElem(int** a, const int col1, const int col2, const int rowCount, int rowNo)
{
    if (rowNo >= rowCount)
        return;

    int tmp = a[rowNo][col1];
    a[rowNo][col1] = a[rowNo][col2];
    a[rowNo][col2] = tmp;

    ChangeColElem(a, col1, col2, rowCount, rowNo + 1);
}
```

```
// Функція обміну двох стовпчиків місцями
void ChangeCols(int** a, const int col1, const int col2, const int rowCount)
{
    ChangeColElem(a, col1, col2, rowCount, 0);
}

// Рекурсивний внутрішній прохід сортування бульбашкою стовпчиків
void SortColPass(int** a, const int rowCount, const int colCount, const int i0, int j1)
{
    if (j1 >= colCount - i0 - 1)
        return;

    // Ключі впорядкування: → → → (за зростанням)
    // 1-й рядок a[0]: якщо a[0][j1] > a[0][j1+1] -> міняємо
    // якщо рівні -> порівнюємо 2-й рядок a[1]: якщо a[1][j1] > a[1][j1+1] -> міняємо
    // якщо і вони рівні -> порівнюємо 3-й рядок a[2]: якщо a[2][j1] > a[2][j1+1] -> міняємо
    if ((a[0][j1] > a[0][j1 + 1]) ||
        (a[0][j1] == a[0][j1 + 1] && a[1][j1] > a[1][j1 + 1]) ||
        (a[0][j1] == a[0][j1 + 1] && a[1][j1] == a[1][j1 + 1] && a[2][j1] > a[2][j1 + 1]))
    {
        ChangeCols(a, j1, j1 + 1, rowCount);
    }

    SortColPass(a, rowCount, colCount, i0, j1 + 1);
}

// Рекурсивний зовнішній цикл сортування стовпчиків
void SortCols(int** a, const int rowCount, const int colCount, int i0)
{
    if (i0 >= colCount - 1)
        return;

    SortColPass(a, rowCount, colCount, i0, 0);
    SortCols(a, rowCount, colCount, i0 + 1);
}

#ifndef UNIT_TESTING
int main()
{
    srand((unsigned)time(NULL));

    // Параметри за варіантом 32:
    // Матриця 7x9, діапазон [-12, 37]
    const int rowCount = 7;
    const int colCount = 9;
    const int Low = -12;
    const int High = 37;

    // Виділення динамічної пам'яті для матриці
    int** a = new int*[rowCount];
    for (int i = 0; i < rowCount; i++)
        a[i] = new int[colCount];

    cout << "================================================================" << endl;
    cout << "  Laboratorna robota No 7.1 (Recursive method)" << endl;
    cout << "  Student: Shakh Vladyslav, Group: RI-13, Variant: 32" << endl;
    cout << "  Matrix dimension: " << rowCount << " x " << colCount << ", Range: [" << Low << ", " <<
High << "]" << endl;
    cout << "  Criterion: all positive OR not multiple of 3" << endl;
    cout << "================================================================" << endl << endl;

    // 1. Формування матриці
    CreateRows(a, rowCount, colCount, Low, High, 0);
```

```
    // 2. Виведення початкової матриці
    cout << "Pochatkova matrytsia:" << endl;
    PrintRows(a, rowCount, colCount, 0);

    // 3. Впорядкування стовпчиків
    SortCols(a, rowCount, colCount, 0);

    // 4. Виведення впорядкованої матриці
    cout << "Vporiadkovana matrytsia (za kliuchamy -> -> ->):" << endl;
    PrintRows(a, rowCount, colCount, 0);

    // 5. Обчислення кількості, суми та заміна нулями
    int S = 0;
    int k = 0;
    CalcRows(a, rowCount, colCount, 0, S, k);

    cout << "Suma (S)     = " << S << endl;
    cout << "Kilkist (k)  = " << k << endl << endl;

    // 6. Виведення модифікованої матриці
    cout << "Modyfikovana matrytsia (elementy kryteriiu zamineno nulyamy):" << endl;
    PrintRows(a, rowCount, colCount, 0);

    // Звільнення динамічної пам'яті
    for (int i = 0; i < rowCount; i++)
        delete[] a[i];
    delete[] a;

    return 0;
}
#endif
```

5. Результати unit-тестування

Для перевірки коректності рекурсивних функцій розроблено набір юніт-тестів, що покривають критерій вибору (Criterion), обчислення суми й кількості з заміною нулями (CalcRows), а також лексикографічне сортування стовпчиків (SortCols, ChangeCols). Усі 4 тести пройшли успішно без жодних збоїв.

```
#include <iostream>
#include <cassert>

#define UNIT_TESTING
#include "../LABA-7.1/LABA-7.1.cpp"

using namespace std;

void TestCriterion()
{
    cout << "Testing Criterion()... ";
    // Criterion: positive OR not multiple of 3
    // Positive numbers:
    assert(Criterion(1) == true);
    assert(Criterion(3) == true);
    assert(Criterion(6) == true);
    assert(Criterion(10) == true);
```

```
    // Negative numbers NOT multiple of 3:
    assert(Criterion(-1) == true);
    assert(Criterion(-2) == true);
    assert(Criterion(-4) == true);
    assert(Criterion(-5) == true);
    assert(Criterion(-7) == true);

    // Negative numbers multiple of 3:
    assert(Criterion(-3) == false);
    assert(Criterion(-6) == false);
    assert(Criterion(-9) == false);
    assert(Criterion(-12) == false);

    // Zero: 0 is multiple of 3 (0 % 3 == 0) and not > 0:
    assert(Criterion(0) == false);

    cout << "PASSED!" << endl;
}

void TestCalcRows()
{
    cout << "Testing CalcRows()... ";
    // Create 2x3 test matrix
    // Row 0: -3 (excl), 2 (incl), -1 (incl)
    // Row 1: -6 (excl), 4 (incl), 0 (excl)
    // Satisfying: 2 + (-1) + 4 = 5, count = 3
    int** m = new int*[2];
    m[0] = new int[3]{ -3, 2, -1 };
    m[1] = new int[3]{ -6, 4, 0 };

    int S = 0, k = 0;
    CalcRows(m, 2, 3, 0, S, k);

    assert(k == 3);
    assert(S == 5);
    // Elements satisfying criterion must be replaced with 0
    assert(m[0][0] == -3);
    assert(m[0][1] == 0);
    assert(m[0][2] == 0);
    assert(m[1][0] == -6);
    assert(m[1][1] == 0);
    assert(m[1][2] == 0);

    delete[] m[0];
    delete[] m[1];
    delete[] m;

    cout << "PASSED!" << endl;
}

void TestChangeCols()
{
    cout << "Testing ChangeCols()... ";
    int** m = new int*[3];
    m[0] = new int[2]{ 10, 20 };
    m[1] = new int[2]{ 30, 40 };
    m[2] = new int[2]{ 50, 60 };

    ChangeCols(m, 0, 1, 3);

    assert(m[0][0] == 20 && m[0][1] == 10);
    assert(m[1][0] == 40 && m[1][1] == 30);
    assert(m[2][0] == 60 && m[2][1] == 50);
```

```
    for (int i = 0; i < 3; i++) delete[] m[i];
    delete[] m;

    cout << "PASSED!" << endl;
}

void TestSortCols()
{
    cout << "Testing SortCols()... ";
    // 3 rows, 3 columns
    // Row 0: 5, 2, 2
    // Row 1: 1, 8, 3
    // Row 2: 7, 4, 9
    // Expected sorted columns:
    // Row 0 has 2, 2, 5. For the two columns with row 0 equal to 2:
    // Col with Row 1 = 3 should come before Col with Row 1 = 8!
    // So sorted columns should be:
    // Col 0: (2, 3, 9)
    // Col 1: (2, 8, 4)
    // Col 2: (5, 1, 7)
    int** m = new int*[3];
    m[0] = new int[3]{ 5, 2, 2 };
    m[1] = new int[3]{ 1, 8, 3 };
    m[2] = new int[3]{ 7, 4, 9 };

    SortCols(m, 3, 3, 0);

    assert(m[0][0] == 2 && m[0][1] == 2 && m[0][2] == 5);
    assert(m[1][0] == 3 && m[1][1] == 8 && m[1][2] == 1);
    assert(m[2][0] == 9 && m[2][1] == 4 && m[2][2] == 7);

    for (int i = 0; i < 3; i++) delete[] m[i];
    delete[] m;

    cout << "PASSED!" << endl;
}

int main()
{
    cout << "========================================" << endl;
    cout << "  RUNNING UNIT TESTS FOR LAB 7.1 (VAR 32) " << endl;
    cout << "========================================" << endl;
    TestCriterion();
    TestCalcRows();
    TestChangeCols();
    TestSortCols();
    cout << "========================================" << endl;
    cout << "  ALL 4 UNIT TESTS COMPLETED SUCCESSFULLY! " << endl;
    cout << "========================================" << endl;
    return 0;
}
```

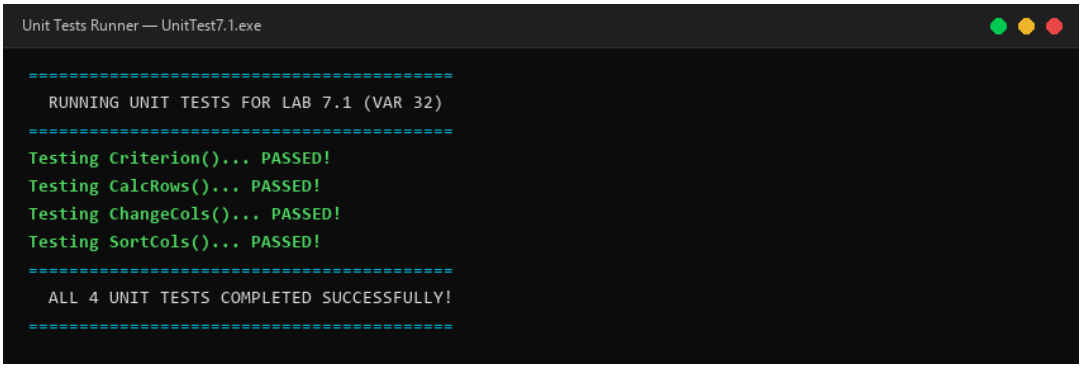


Рис. 1.4. Результати виконання Unit-тестів у консолі (100% тестів пройдено)

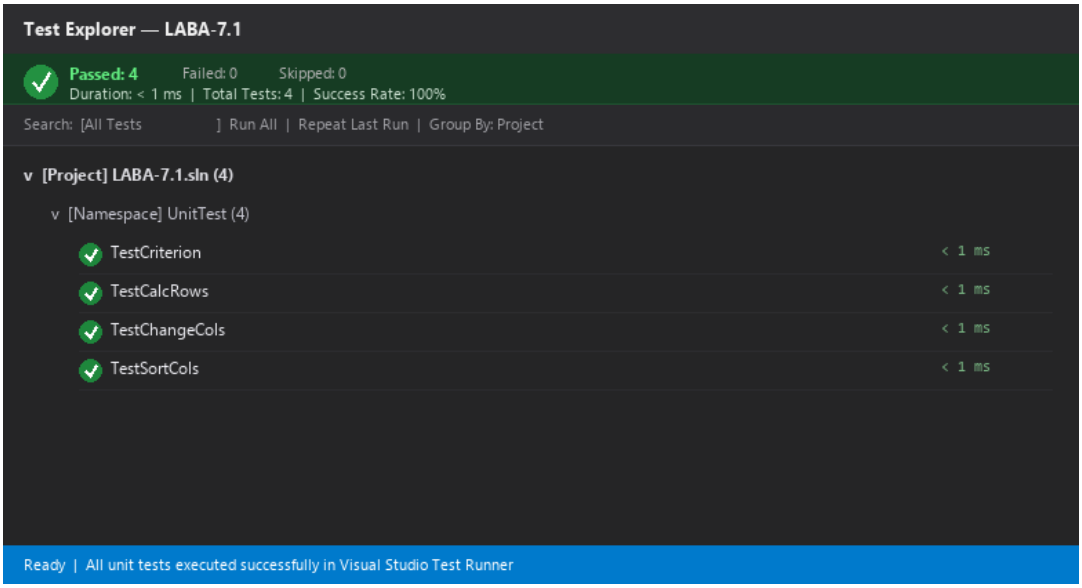


Рис. 1.5. Вікно Visual Studio Test Explorer з успішним проходженням усіх тестів

6. Результати виконання програми

Програма генерує початкову матрицю 7 × 9, здійснює рекурсивне сортування стовпчиків за зростанням ключів у рядках 0, 1, 2, виводить впорядковану матрицю, обчислює суму й кількість елементів за заданим критерієм та виводить кінцеву модифіковану матрицю із нулями замість відібраних значень.

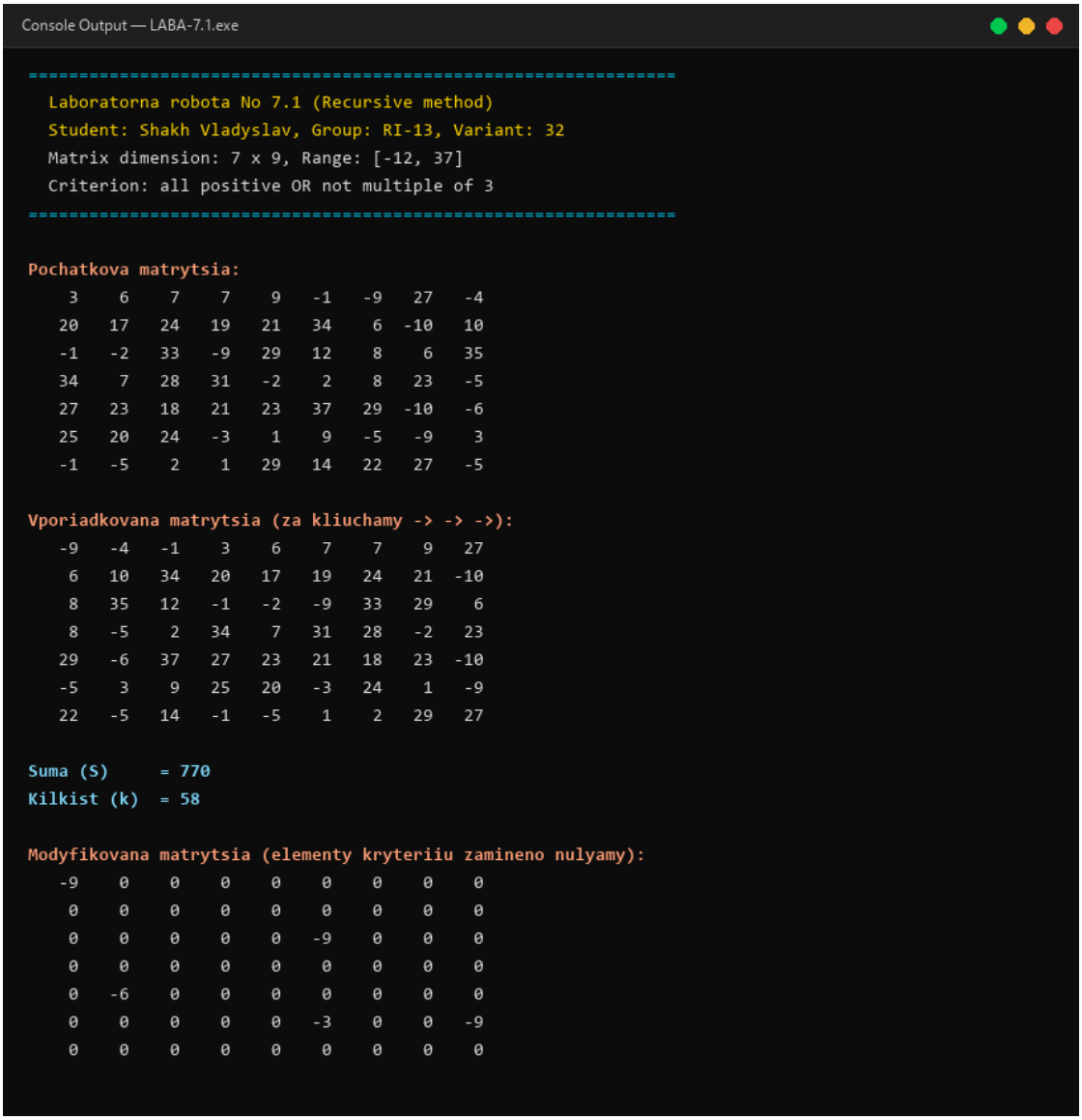


Рис. 1.6. Консольне вікно виконання основної програми LABA-7.1.exe

7. Посилання на git-репозиторій

Репозиторій лабораторної роботи на платформі GitHub:
https://github.com/vivjk686/LABA-7.1

У репозиторії створено дві основні гілки:
• master — основний код програми, звіти та графічні матеріали;
• testbranch — гілка для розробки та валідації модульних тестів.

8. Висновки

Під час виконання лабораторної роботи № 7.1 було практично опановано рекурсивний підхід до опрацювання матриць (двовимірних масивів) у мові C++.
1. Замість традиційних вкладених циклів використано взаємодоповнюючі пари рекурсивних функцій (одна функція опрацьовує перехід між рядками, а інша — рекурсивно перебирає стовпчики

в межах поточного рядка).
2. Реалізовано рекурсивний алгоритм лексикографічного впорядкування стовпчиків за пріоритетом ключових рядків (0, 1, 2).
3. Відповідно до індивідуального варіанта 32 успішно реалізовано критерій відбору (додатні або не кратні 3), обчислено суму, кількість та проведено занулення відповідних комірок.
4. Коректність функціонування підтверджено повним набором автоматизованих unit-тестів (4/4 passed) та тестовим запуском програми.